BPE: A Lightweight Native BPMN Implementation Preserving Clean Process Semantics

Maxim Sokhatsky
Synrc Research

May 22, 2026

Abstract

BPE (Business Process Engine) is a compact, production-grade workflow engine written in Erlang/OTP that implements a clean subset of BPMN 2.0. Unlike traditional BPMN engines such as Camunda, where developers are forced to attach scripts and listeners to gateways and sequence flows (pre/post events), BPE treats the process as a natural sequence of actions executed within a single semantic step. This approach dramatically reduces complexity while maintaining full persistence, fault-tolerance, and auditability.

1 Introduction

BPE is a lightweight business process management engine developed as part of the Synrc stack. It is designed for real enterprise use — particularly in banking, document management, CRM, and telecom systems — where reliability, auditability, and simplicity are paramount.

At its core, BPE is a microkernel that executes native Erlang record-based process definitions combined with Event-Condition-Action logic. The entire engine consists of roughly 400 lines of clean functional code, making it one of the most compact and maintainable workflow engines available.

BPE stores all process state and history in KVS (Key-Value Store), providing strong persistence guarantees and seamless integration with the Erlang ecosystem. Every process instance is a supervised Erlang process with full crash recovery and migration capabilities.

2 Key Design Philosophy

The central idea of BPE is to preserve **clean process semantics**.

In most BPMN engines (e.g. Camunda, Activiti, jBPM), the execution model forces developers to split business logic across:

- Sequence flow conditions
- Execution listeners (pre/post)
- Task listeners
- Gateways with attached scripts

This results in fragmented logic that is hard to read, debug, and maintain.

BPE takes the opposite approach: **the process flow is executed naturally in one action**. A task (userTask or serviceTask) defines its module and the engine calls the corresponding Erlang functions directly. The transition to the next step is handled atomically within the same semantic context.

This design aligns with Erlang's native process calculus (Pi-calculus) semantics and provides a much more idiomatic and readable way to express business workflows.

3 Core Concepts

3.1 Process Definition

Processes are defined as Erlang records:

```
deposit_app() -> #process {
    name = 'Create Deposit Account',
    flows = [
        #sequenceFlow{source='Init',      target='Payment'},
        #sequenceFlow{source='Payment',   target='Signatory'},
        #sequenceFlow{source='Payment',   target='Process'},
        #sequenceFlow{source='Signatory', target='Process'},
        #sequenceFlow{source='Signatory', target='Finish'}
    ],
    tasks = [
        #userTask    {name='Init',      module=deposit},
        #userTask    {name='Signatory', module=deposit},
        #serviceTask {name='Payment',   module=deposit},
        #serviceTask {name='Process',   module=deposit},
        #endEvent    {name='Finish'}
    ],
    beginEvent = 'Init',
    endEvent = 'Final'
}.
```

3.2 Main API

- `bpe:start(Process, Options)` — starts a new process instance
- `bpe:complete(PidOrId)` — moves the process to the next step
- `bpe:amend(PidOrId, Document)` — attaches business documents
- `bpe:event(Event)` — sends external events
- `bpe:history(Id, N)` — retrieves execution log
- `bpe:load(Id)` — loads a terminated process for continuation

All operations are atomic and persisted through KVS.

4 Comparison with Camunda and Other Engines

- **Clean Semantics**: BPE executes business logic inside task modules rather than scattering it across listeners and gateways.
- **No XML Hell**: Processes are defined in Erlang terms (or optional DSLs).
- **Native Persistence**: Every step and document is automatically saved via KVS/RocksDB.
- **Supervision & Fault Tolerance**: Built on Erlang/OTP principles.
- **Auditability**: Complete immutable history is maintained automatically.
- **Performance**: Extremely low overhead — suitable for high-frequency financial workflows.

While Camunda excels in visual modeling for large analyst teams, BPE is optimized for engineering teams that value clarity, maintainability, and performance over drag-and-drop modeling.

5 Use Cases

BPE has been successfully used in:

- Banking core systems (deposit, credit, payment processing)
- Document workflow and approval systems
- CRM interaction flows
- Telecom service activation and billing processes
- Regulatory compliance workflows

Its ability to handle both human tasks and automated service tasks within the same clean model makes it particularly suitable for hybrid human–machine business processes.

6 Conclusion

BPE demonstrates that a BPMN-compatible workflow engine can be implemented in a remarkably clean and efficient way by staying close to the underlying virtual machine semantics. By executing sequence flows naturally within one action and leveraging Erlang's process model, BPE achieves excellent readability, strong fault tolerance, and production readiness with minimal code.

For projects that prioritize engineering clarity and operational reliability over visual designer tools, BPE offers a compelling lightweight alternative to heavyweight BPM suites.

Documentation and source code: https://synrc.com